

Single-Machine Optimization of Perturbation-Based Loss-Grid Computation

Given Name Surname

Department of Advanced Computing Sciences

Faculty of Science and Engineering

Maastricht University

Maastricht, The Netherlands

Abstract—TODO left for last

I. INTRODUCTION

A loss landscape is a 2D scalar field made by perturbing a trained parameter vector along two directions and evaluating the loss at each point [2], supporting generalization analysis and interactive model-comparison tools [3], [4]. Computationally, the landscape is represented as an array of scalar loss evaluations indexed by grid coordinates. Each entry evaluates the model at the corresponding perturbed parameter vector on data. One landscape can require hundreds to thousands of such evaluations [2], [11]. In interactive checkpoint comparison [3], this repeated work can dominate latency on commodity hardware.

The problem is to evaluate candidate single-machine optimization mechanisms for perturbation-based loss-grid computation under the constraint of one GPU, the available CPU cores, and finite RAM and VRAM. The optimization target is the construction of the loss grid itself: the perturbed model evaluations that fill the grid. Our thesis explores three mechanisms, each posed as a research question. The first is single-device evaluation improvement: **RQ1. Do PyTorch’s stateless evaluation APIs reduce perturbation-based loss-grid runtime, and for which architecture types?** The second is heterogeneous CPU+GPU scheduling: **RQ2. Does heterogeneous CPU+GPU scheduling reduce perturbation-based loss-grid runtime, and does the CPU/GPU throughput ratio determine where it applies across architecture types?** The third is calibrated policy reuse across checkpoint sessions: **RQ3. Can the calibration step be included in a related checkpoint session and still reduce total session time when the calibrated policy is reused across checkpoints?**

Because these mechanisms depend on hardware and workload, the study determines where RQ1 and RQ2 produce statistically supported runtime reductions, where they regress or lose practical utility, and whether all measured configurations preserve the numerical equivalence of the resulting loss surface. RQ3 then asks whether the one-time calibration cost can be amortized under a multi-checkpoint session.

Our thesis was prepared in partial fulfilment of the requirements for the Degree of Bachelor of Computer Science, Maastricht University. Supervisor(s): Max Sondag, Georgios Stamoulis

A. Structure of the Report

Section II frames our thesis with respect to prior work. Section III defines the baseline and the three optimization approaches (one per RQ). Section IV gives the experimental design, workloads, platforms, and measurement protocol. Section V reports measurements; Section VI interprets them as applicability conditions; Section VII answers the RQs.

B. Contributions

(RQ1) An applicability study of PyTorch stateless and vectorized evaluation primitives (`functional_call+vmmap`, `torch.compile`) for loss-grid construction across architectures and backends.

(RQ2) An applicability study of heterogeneous CPU+GPU scheduling for single-machine loss-grid computation, with CPU worker count fixed at maximum to isolate the CPU/GPU inference-throughput ratio as the explanatory variable across architectures.

(RQ3) An evaluation of whether a calibrated policy over the worker-count and batch-size axes that RQ2 holds fixed can be reused across related checkpoint-comparison sessions to amortize calibration cost. RQ1 and RQ2 provide the architecture-dependent evidence; RQ3 inherits a hybrid-affinity regime from RQ2, native or slowed, where calibration has an opportunity to change session time.

II. RELATED WORK

Single-machine loss-grid optimization intersects loss-landscape visualization, adaptive loss-grid sampling, heterogeneous CPU/GPU scheduling, and AEOS-style autotune-and-cache patterns.

A. Loss-landscape visualization and analytics

Our thesis takes the baseline from Li et al. [2], who introduced a 2D filter-normalized loss grid extending the 1D-interpolation approach of Goodfellow et al. [1]. The official implementation [11] uses MPI to distribute grid-point evaluation across multiple GPUs, on one or more nodes. LossLens [3], [4] integrated this construct into a multi-scale loss-landscape visual-analytics workflow; comparing independently trained model instances is one of its diagnostics. Within the reviewed literature we found no evaluation of whether `functional_call+vmmap` or `torch.compile` transfer

to the loss-grid pattern (one model, many perturbations), or of how applicability varies across architectures.

B. Adaptive and progressive loss-grid sampling

A different direction replaces the canonical full grid with an adaptive sampler that evaluates loss only at points predicted to carry visual information. A non-peer-reviewed course report by Prakash et al. [5] applies this to loss-landscape visualization, replacing regular α, β sampling with an adaptive algorithm focused on “interesting” regions. We considered this direction but do not pursue it; our thesis keeps the grid fixed and optimizes its computation, so adaptive sampling is related work outside the evaluated mechanisms.

C. Heterogeneous compute and ML inference throughput

Task-parallel heterogeneous scheduling assigns independent tasks across devices. In the simplest variant, idle workers poll a shared queue and claim one task at a time. For this primitive, chunk size trades load balance against dispatch overhead [6]. More complex policies include HEFT-style cost-model scheduling [26] and priority-based work-stealing, exemplified by StarPU [7]; these address load imbalance and per-task cost variance.

An orthogonal approach splits a single computation across devices, with per-device stages composing a pipeline. OpenVINO’s HETERO execution mode is a production example at the intersection of heterogeneous compute and ML inference [8]: the inference model is divided into stages, each stage is assigned to a device, and the output of one stage is fed as input to the next.

Adaptive request batching to meet a latency target is a standard technique in ML inference serving [24]. OpenVINO’s automatic batching [9] groups concurrent inference requests at runtime to increase device utilization (on supported devices). At implementation level, auto-batching operates on the input/request batch dimension of one fixed compiled model: requests are packed into slices of a larger input/output tensor and executed by a reshaped batched model [10]. The cited systems do not evaluate whether the CPU/GPU inference-throughput ratio determines where heterogeneous scheduling reduces loss-grid runtime; we adopt the ratio as a working assumption, since a CPU with throughput negligible against the GPU cannot reduce grid runtime under any scheduling policy.

D. Autotune-and-cache patterns

ATLAS [22] introduces AEOS (Automated Empirical Optimization of Software): empirically time candidate implementations of BLAS inner loops (GEMM, DAXPY, etc.) on the target machine, select the fastest variant for the machine’s cache and register hierarchy, and persist the selection so it is reused without re-tuning. The same empirical autotune-and-cache pattern appears in modern deep-learning compilers: TVM’s AutoTVM tunes tensor-operator implementations with an ML cost model and caches the selected schedule for reuse [25].

In a shared-queue CPU/GPU scheduler for ML-inference loss-grid, effective CPU throughput can vary with CPU worker count and CPU batch size. Then the optimal policy is per-(machine, workload), so a calibration step is needed before policy reuse can amortize. We lift the ATLAS signature from BLAS operations to (machine, workload) for this case. Within the reviewed literature we found no evaluation of whether the calibration step that shared-queue policy selection requires amortizes across interactive checkpoint sessions.

E. Where our thesis sits

Prior work provides the core algorithm [2], [11] and its interactive model-comparison use case [3], [4], the pure self-scheduling primitive [6] and the broader heterocompute-scheduler family [7] from HPC/scientific workloads, heterocompute implemented as model-parallel and request-batching in ML inference [8], [9], the AEOS pattern [22], and the adjacent adaptive-sampling direction [5]. StarPU, OpenVINO, and ATLAS are design precedents for our scheduler and calibration cache, not direct baselines we benchmark against.

Available loss-grid implementations [3], [11] target distributed or multi-GPU scaling. The gap is whether fixed-grid, perturbation-based loss-grid computation can be optimized on one commodity machine with one GPU, available CPU cores, finite RAM, and finite VRAM. Our thesis studies single-device evaluation, CPU/GPU scheduling, and the integration of calibration reuse into interactive checkpoint sessions. The gaps that motivate each of these directions are stated in the respective subsections above; among the cited systems, none directly targets the workload shape we address (one model family, many perturbations of one checkpoint, shared data).

III. METHODS

This section defines the baseline, the three optimization approaches mapped to RQ1–RQ3, and the shared measurement and surface-validation rules. RQ3 asks whether the calibration routine can be included in a multi-checkpoint comparison session and still amortize when the selected policy is reused. Experiment-specific parameters are stated in Section IV.

A. Canonical Loss-Grid Algorithm (Baseline)

We implement the baseline as a sequential GPU-only loop over the 2D filter-normalized grid of Li et al. [2], [11]. For grid coordinate (α, β) it evaluates $w(\alpha, \beta) = w_0 + \alpha d_1 + \beta d_2$ on a fixed data subset, writes the scalar loss into the surface, and repeats until the grid is complete. The directions d_1, d_2 use the upstream filter-normalized `weights` mode.

The baseline is a single-machine GPU-only implementation that visits grid points sequentially. It excludes MPI, distributed execution, and CPU helper work. This gives RQ2 a fixed denominator: the canonical GPU-only path. Per-grid-point semantics are unchanged; all comparisons in Section V are relative to this baseline. A *perturbation* is a single parameter context $w(\alpha, \beta)$ of a checkpoint; *checkpoint* denotes a distinct trained model.

TABLE I
ALGORITHM-LEVEL OPTIMIZATION CANDIDATES.

Candidate	Components	Method role
baseline	mutation + sequential loop	reference implementation
vmapped	stateless call + vmap + chunking	vectorized forward/loss over perturbations
compiled	compile baseline eval path	graph capture and kernel fusion on per-grid-point work
compiled vmapped	compile vmapped candidate	graph capture over vectorized point evaluation

B. Algorithm-Level Optimization Candidates (RQ1)

The canonical loop evaluates many perturbed parameter states against the same data subset. RQ1 tests whether PyTorch’s documented `functional_call+vmap` ensembling pattern [12], [13], [16] and `torch.compile` [14] apply to this workload pattern (Table I). These two candidates match the single-machine constraint. The loss grid evaluates one model under many parameter states on shared data, which is the case PyTorch’s ensembling pattern targets [16], and graph capture is the relevant kernel-level optimization for a fixed evaluation path. Both stay within one GPU and add no devices, which keeps the algorithm-level question (RQ1) separate from the scheduling question (RQ2).

Each candidate is first compared with `vanilla` on T_{grid} . Let $s_v = T_b/T_v$, $s_c = T_b/T_c$, and $s_{vc} = T_b/T_{vc}$ for `vmapped`, `compiled`, and `compiled_vmapped`. These three baseline-relative speedups give the per-candidate verdicts. For composition, the ratio

$$q_{\text{compose}} = \frac{s_{vc}}{\max(s_v, s_c)}$$

compares the composed candidate against the better individual transform. All four ratios use the CI verdict taxonomy in Section III-E.

1) *Vectorized Point Evaluation with `torch.vmap`*: `functional_call` supplies a perturbed parameter map to the model without rewriting the module object. `vmap` then maps the loss over a leading dimension of K stacked perturbations, evaluating a chunk of perturbations against each data batch. Chunking bounds memory; the expected runtime reduction is dispatch amortization and backend batching, but the result is backend- and architecture-dependent.

2) *Graph Capture with `torch.compile`*: Graph capture is tested in two placements. `compiled` wraps the baseline point-evaluation path with `torch.compile` while leaving the grid loop unchanged. `compiled_vmapped` applies the same capture step to the batched evaluator, testing whether it accelerates or slows vectorized point evaluation on the loss-grid pattern. Both placements set `torch.compile`’s `fullgraph` parameter, so the captured region either compiles as one graph or the build fails. This prevents silent eager fallback inside the measured region, so a measured speedup

reflects fully compiled execution. PyTorch 2 reports training-time speedups from this compilation path [23]; the loss-grid setting is forward+loss inference with no backward pass, so whether those gains transfer is an empirical question.

C. Heterogeneous Scheduling (RQ2)

RQ2 uses a shared-queue self-scheduler. A task is one grid-point evaluation of the canonical algorithm. This work is uniform, so the load-imbalance cost of the chunk-size trade-off (Section II-C) is none and chunk size one is used. CPU and GPU workers poll the queue and claim one grid point per task. Both devices run the same forward+loss code and share the baseline surface-validation contract. The CPU/GPU throughput ratio r is taken as the explanatory variable for where heterogeneous execution helps. Here r is CPU forward+loss throughput divided by GPU forward+loss throughput under the same workload, data subset, and batch policy. RQ2 fixes $p = p_{\text{max}}$ (max available cores) to isolate the throughput-ratio question. RQ3 treats p as a calibration parameter because session reuse must select an actual policy.

The `gpu_slowdown_factor` is added latency on the GPU worker path that artificially tips the achieved CPU/GPU inference-throughput ratio toward parity when native GPU throughput is too high. r_{native} is r measured without slowdown. The slowdown is implemented as a sleep on the GPU worker task path that scales by s , where $s = 1/r_{\text{native}}$ when $r_{\text{native}} < 1$ and $s = 1.0$ otherwise. The queue-based scheduler sees only task-completion timing, so the sleep is sufficient at this granularity. Slowed runs explore controlled CPU/GPU throughput ratios; their speedups show where adding workers stops helping and are not gains available on the unslowed machine.

D. Calibration Cache (RQ3)

RQ3 measures whether the one-time cost of selecting a CPU/GPU scheduling policy can be amortized in a multi-checkpoint session (as in LossLens [3]). A related checkpoint shares architecture, dataset, loss, and grid spec, but differs in learned weights. Under the intended workflow, the calibration step is unconditional: it runs before the session and may select either a cached hybrid policy or the GPU-only baseline. If calibration resolves to GPU-only, heterogeneous compute is not used and the default path is chosen. The calibration cost is then useless for that row, so amortization fails. In RQ3, amortization means the `cached_composed` (calibrate once, reuse the selected policy) session remains faster than `vanilla` after calibration time is included.

The GPU-side implementation is controlled by `rq3_config`. This configuration is fixed for the RQ3 run and is determined by RQ1 for the selected workload/platform pair. Calibration is conditioned on this fixed GPU-side configuration because changing GPU-side time changes the CPU/GPU break-even point.

Calibration selects a scheduling policy tuple $c = (\pi, b_g, p, b_c)$: $\pi \in \{\text{gpu_only}, \text{gpu_cpu_hybrid}\}$ is the placement policy, b_g is the GPU batch size, p is the CPU

worker count, and b_c is the CPU batch size. The tuple includes p and b_c because CPU forward+loss throughput is not assumed monotone in worker count or batch size, so calibration sweeps these axes empirically. All four fields are swept together; in `gpu_only` mode p and b_c are ignored.

Calibration uses a square probe grid of side m , the smallest integer with $m^2 \geq 4(1 + p_{\max})$, where $p_{\max}$ is the number of CPU cores available to the Python process. The search stops after `calibration_retry` = P consecutive non-improvements. The selected tuple is the fastest valid hybrid configuration, or `gpu_only` if no hybrid tuple beats the matched GPU-only configuration.

Cache reuse runs an empirical tuning step and stores the result in a lookup table with key (machine, architecture, dataset, loss, grid spec, operating point (native or slowed), `rq3_config`); the session tunes once at the start, caches the selected tuple under this key, reuses it across matching lookups within the session, and scopes the cached result to the machine.

Amortization is evaluated at the selected RQ3 operating point with three metrics. **Session speedup** is `vanilla` total time divided by `cached_composed` total time at $N = 4$. **Break-even** N^* is $\lceil \text{calibration}_s / (T_v - T_p) \rceil$ when $T_v > T_p$, where T_v is vanilla per-checkpoint time and T_p is per-checkpoint time under the cached policy after calibration. It states how many checkpoints amortize the calibration cost. **Amortization label** is *supported* when speedup > 1.0 and $N^* \leq 4$, *supported asymptotically* when speedup > 1.0 and $N^* > 4$, and *refuted* when speedup ≤ 1.0 or $T_v \leq T_p$. Surface failure on any checkpoint suppresses all three.

E. Benchmarking, Surface Validation, and Uncertainty

Timing and targets. Elapsed time is measured at the call site with `time.perf_counter_ns`; backend completion is synchronized before stopping the timer. T_{grid} measures the wall time to compute one loss grid after inputs are ready (RQ1/RQ2). T_{session} measures a multi-checkpoint session, including one-time calibration when present (RQ3). Each T_{session} value is measured once per (workload, platform, condition), so RQ3 outcomes are descriptive and not CI-bounded. The baseline decomposition $T_{\text{grid}} = T_{\text{perturb}} + T_{\text{bind}} + T_{\text{eval}} + T_{\text{collect}} + \text{residual}$ is computed with the same clock and used only for interpretation. Only T_{perturb} , T_{bind} , and T_{eval} (forward+loss) are timed per grid point; T_{collect} and the residual are reported together as the remainder of T_{grid} .

Uncertainty and verdicts. RQ1 and RQ2 speedup ratios use paired repeated measurements and effect-size confidence intervals [20], [21]. Every such ratio r is expressed with values above 1.0 favoring the candidate. The confidence interval for r gives one of three timing verdicts:

- *supported improvement*: $\text{CI}_{\text{low}} > 1.0$;
- *supported regression*: $\text{CI}_{\text{high}} < 1.0$;
- *unresolved*: the CI contains 1.0.

RQ3 takes one T_{session} measurement per condition, so it does not produce a CI (see amortization-labels in Section III-D).

Pairing and surface validation. Candidate and baseline runs

TABLE II
PLATFORM CHARACTERIZATION

Property	Apple M4	NVIDIA Tesla T4
GPU	M4 integrated	Tesla T4
VRAM	16 GiB (unified)	14.6 GiB (discrete)
CPU cores available	10	2
PyTorch backend	mps	cuda

are paired by workload, seed, grid, and repeat. A timing claim is valid when the candidate surface matches the baseline:

- 1) grid-point count and coordinates match the baseline exactly
- 2) NaN–NaN pairs at matching coordinates count as equal
- 3) finite losses are equal under relative tolerance $1e-5$ with `atol = 0` [15]; this is the `torch.allclose` default relative tolerance with the absolute term removed, loose enough to cover float32 margin of error and tight enough to detect semantic/visual divergence

IV. EXPERIMENTS

This section states the workload set, platforms, and experiment-specific design choices. Shared timing, uncertainty, and surface-validation rules are defined once in Section III-E. The three experiments form a decision pipeline. RQ1 decides which GPU evaluation mode to use and exports it as `rq3_config`. RQ2 decides whether heterogeneous scheduling is worth considering and on which workload/platform regimes yield positive evidence. RQ3 decides whether calibration reuse amortizes only for regimes with positive evidence in RQ2. Each stage depends on the previous result, so a null or negative result upstream narrows the choices available downstream (Sections IV-B–IV-D).

A. Shared Experimental Setup

Platforms. Table II gives the two hardware settings: Apple M4 with unified memory and ten CPU cores, and NVIDIA Tesla T4 with discrete VRAM, higher GPU throughput, and two CPU cores. The unified-vs-discrete memory axis and the CPU-core count separate the RQ2 operating points and bound the `vmap` chunk memory budget in RQ1.

Workload set. A workload is a (dataset, model family, loss function) triple. Four workloads cover distinct computation patterns relevant to the RQ1 transform applicability claim (Table III): `cifar10_resnet20` for 2D convolution (the workload for which CUDA convolution kernels are designed [17]), `cifar10_row_gru` [18], [19] for recurrent row dependencies (no kernel fusion), `mnist_mlp` for MLP image classification, and `california_mlp` for tabular regression. The MLP pair differs in input dimensionality and per-sample compute cost, the axes that should impact `vmap` behavior. These span enough architectures for workload-dependent outcomes to emerge. RQ1 and RQ2 use all four; RQ3 narrows to one case selected by filtering Experiment 2 for hybrid affinity (Section IV-D).

TABLE III
WORKLOAD SET.

Name	Modality	Architecture	Dataset
cifar10_resnet20	image	2D conv (ResNet-20)	CIFAR-10
cifar10_row_gru	sequence	row-wise GRU	CIFAR-10
mnist_mlp	dense image	MLP	MNIST
california_mlp	tabular	MLP	California

Grid and data subset. RQ1 and RQ2 use an 8×8 filter-normalized grid at unit perturbation scale, 1024 evaluation samples per point, and batch size 64. Unit scale matches the upstream default for filter-normalized weights [2], [11]. The 8×8 resolution yields 64 grid-point tasks per run: enough to populate the $K \in \{32, 64\}$ vmap chunks in RQ1 and to fill the shared queue in RQ2, while keeping the full sweep feasible on both platforms. Resolution sets the task count, not the per-point evaluation semantics, so the transform and scheduler verdicts do not depend on it. RQ3 uses a 20×20 session grid matching LossLens’s default resolution [3], plus a calibration probe grid of side m with $m^2 \geq 4(1 + p_{\max})$ (Section III-D).

Repeats and trial order. RQ1 and RQ2 use $R = 5$ paired repeats ($df = R - 1 = 4$, the minimum for the paired t-CI). The paired design removes machine variance between runs, so a small R resolves the per-candidate verdicts across the full 4×2 workload/platform sweep at feasible total cost; larger R tightens the CI with diminishing returns [20]. Within each repeat the candidates run sequentially under a cyclic rotation indexed by the repeat number, so no candidate consistently runs after baseline and inherits its warm cache state [20]. RQ3 takes a single T_{session} measurement per (workload, platform, condition) and is descriptive (Section III-E).

Conditions. *vanilla* denotes the canonical GPU-only path defined in Section III-A. RQ1 pairs each candidate from Section III-B against *vanilla*; RQ2 pairs *hybrid* against *vanilla* under matched GPU slowdown; RQ3 pairs *cached_composed* against *vanilla* run once per checkpoint.

B. Algorithm-Level Optimization (RQ1)

Question. Do PyTorch’s stateless evaluation primitives reduce T_{grid} , and which workload patterns gain or regress?

Hypothesis. Verdicts depend on architecture. We predict supported improvement on the dense MLP rows (the closest match to the vmap ensembling pattern), an unresolved or smaller effect on *cifar10_resnet20* (its convolution kernels already saturate the GPU at this batch size), and an unresolved or supported regression on *cifar10_row_gru* (its inner row-by-row dependence is not removable by batching perturbations).

Design. The candidates from Section III-B run on each workload/platform pair under the shared setup. vmap-bearing candidates sweep $K \in \{32, 64\}$ as a memory-vs-throughput sweep: the smaller chunk bounds memory and isolates the dispatch-amortization gain, the larger chunk tests whether

higher per-chunk throughput exceeds the added memory pressure. The composed candidate is compared against the better individual transform via q_{compose} (Section III-B).

C. Hybrid Scheduler Applicability (RQ2)

Question. Does heterogeneous CPU+GPU scheduling reduce T_{grid} , and does the CPU/GPU forward+loss throughput ratio r determine where it applies?

Hypothesis. Near parity, dynamic self-scheduling [6] absorbs useful CPU work and reduces T_{grid} against the matched GPU-only path. Far below parity, GPU-only dominates or the result is unresolved. We also expect more headroom on the discrete-memory T4 than on the unified-memory M4, since unified memory shares one bandwidth budget between CPU and GPU (Section VI-B).

Design. Native r on the available hardware does not span the regimes where the ratio can change the outcome, so the GPU slowdown factor sweeps r to find the hybrid applicability threshold (Section III-C); the queue-based scheduler sees only task-completion timing, so the sleep is sufficient at this granularity. Each workload/platform pair first records r_{native} ; if $r_{\text{native}} < 1$, the slowdown factor $s = 1/r_{\text{native}}$ targets parity, otherwise $s = 1$. At the achieved ratio, *hybrid* is paired against *vanilla* with the same slowdown applied to the GPU side. The GPU path remains *vanilla*, so the measured effect belongs to the scheduler axis.

Native rows and slowed rows answer different questions. The native- r row runs with no added latency, so it is the only row claiming a speedup realizable on this hardware. The slowed rows distort r to locate where adding CPU workers stops helping, without claiming a gain on this machine. Table VII keeps native r and achieved r in separate columns so the two are not conflated.

D. Calibration Cache Amortization (RQ3)

Question. When the unit of evaluation changes from one grid to a related-checkpoint session, can the unconditional calibration step amortize via cached-policy reuse?

Motivation. A LossLens-style session [3] evaluates several related checkpoints in succession. Hybrid scheduling’s applicability varies by machine and workload (RQ2), so a session cannot commit to hybrid statically. It must calibrate, and calibration is allowed to select GPU-only as a valid outcome. The question is whether the one-time calibration cost is recovered over the session. RQ3 is a scoped measurement: it uses one favorable regime and a single session measurement per condition, so it supports or refutes amortization for that case and does not establish a general amortization claim across other workloads or machines.

Hypothesis. Amortization holds where Experiment 2 gives positive hybrid evidence and the per-checkpoint hybrid saving exceeds the calibration cost amortized over $N = 4$.

Design. RQ3 filters Experiment 2 for a (workload, regime) with hybrid affinity, meaning a supported hybrid win. It prefers a native regime, where the win is a practical-hardware claim, and falls back to a slowed regime, where the win is

a controlled demonstration. The selected regime’s slowdown is inherited as a single operating point and applied uniformly to calibration, `vanilla`, and `cached_composed`, so the comparison is fair within that regime. Each session contains $N = 4$ checkpoints from one training trajectory, drawn at evenly spaced epochs (the exact spacing recorded with the asset manifest). The GPU-side configuration is `rq3_config`, selected by rule: the RQ1 candidate with the highest supported speedup at the workload/platform pair, taking the smaller K on ties for `vmap`-bearing candidates, and falling back to baseline when no RQ1 candidate has a supported improvement. Two conditions are compared: `vanilla` run once per checkpoint, and `cached_composed`, which calibrates on checkpoint 0 and reuses the selected tuple for all checkpoints under `rq3_config`. Amortization here means `cached_composed` has session speedup > 1.0 after calibration time is included; the verdict is labeled native or slowed, and break-even N^* follows Section III-D.

Two upstream outcomes shape RQ3. If RQ1 gives no supported improvement at the selected pair, `rq3_config` falls back to baseline (Section III-D), so RQ3 still runs but over the GPU-only evaluation path. If Experiment 2 shows hybrid affinity in no regime, native or slowed, there is no regime to inherit, and RQ3 reports amortization refuted without running a session because calibration cannot amortize anywhere. Within a selected regime, calibration may still resolve to GPU-only, which also refutes amortization. RQ3 therefore results in either a regime where reuse amortizes or evidence that it does not.

V. RESULTS

This section reports measurements against the timing-verdict taxonomy of Section III-E. Tables are populated from recorded runs; no rows failed the surface validation.

A. Motivation: Baseline Section Decomposition

TABLE IV
BASELINE SECTION TIME SHARES (%) OF T_{GRID} ON `VANILLA_BASELINE`. PERTURB., BINDING, AND BATCH EVAL. ARE TIMED PER GRID POINT; THE REMAINDER (COLLECT AND RESIDUAL) IS NOT SHOWN.

Platform	Workload	Total (s)	Perturb. (%)	Binding (%)	Batch eval. (%)
MPS	cifar10_resnet20	–	–	–	–
	cifar10_row_gru	–	–	–	–
	california_mlp	–	–	–	–
	mnist_mlp	–	–	–	–
		–	–	–	–
T4	cifar10_resnet20	–	–	–	–
	cifar10_row_gru	–	–	–	–
	california_mlp	–	–	–	–
	mnist_mlp	–	–	–	–
		–	–	–	–

B. Experiment 1: Algorithm Redesign

TABLE V
CANDIDATE SPEEDUP ON MPS RELATIVE TO `VANILLA_BASELINE`. VERDICT PER SECTION III-E.

Workload	Candidate	Mean speedup	95% CI
cifar10_resnet20	vmapped ($K = 32$)	–	–
	vmapped ($K = 64$)	–	–
	compiled	–	–
	compiled_vmapped ($K = 32$)	–	–
	compiled_vmapped ($K = 64$)	–	–
cifar10_row_gru	vmapped ($K = 32$)	–	–
	vmapped ($K = 64$)	–	–
	compiled	–	–
	compiled_vmapped ($K = 32$)	–	–
	compiled_vmapped ($K = 64$)	–	–
california_mlp	vmapped ($K = 32$)	–	–
	vmapped ($K = 64$)	–	–
	compiled	–	–
	compiled_vmapped ($K = 32$)	–	–
	compiled_vmapped ($K = 64$)	–	–
mnist_mlp	vmapped ($K = 32$)	–	–
	vmapped ($K = 64$)	–	–
	compiled	–	–
	compiled_vmapped ($K = 32$)	–	–
	compiled_vmapped ($K = 64$)	–	–

TABLE VI
CANDIDATE SPEEDUP ON T4 RELATIVE TO `VANILLA_BASELINE`. SAME TIMING-VERDICT TAXONOMY AS TABLE V.

Workload	Candidate	Mean speedup	95% CI
cifar10_resnet20	vmapped ($K = 32$)	–	–
	vmapped ($K = 64$)	–	–
	compiled	–	–
	compiled_vmapped ($K = 32$)	–	–
	compiled_vmapped ($K = 64$)	–	–
cifar10_row_gru	vmapped ($K = 32$)	–	–
	vmapped ($K = 64$)	–	–
	compiled	–	–
	compiled_vmapped ($K = 32$)	–	–
	compiled_vmapped ($K = 64$)	–	–
california_mlp	vmapped ($K = 32$)	–	–
	vmapped ($K = 64$)	–	–
	compiled	–	–
	compiled_vmapped ($K = 32$)	–	–
	compiled_vmapped ($K = 64$)	–	–
mnist_mlp	vmapped ($K = 32$)	–	–
	vmapped ($K = 64$)	–	–
	compiled	–	–
	compiled_vmapped ($K = 32$)	–	–
	compiled_vmapped ($K = 64$)	–	–

C. Experiment 2: Hybrid Scheduler Applicability

TABLE VII
HYBRID SCHEDULER APPLICABILITY. r IS THE CPU/GPU THROUGHPUT RATIO; *achieved* r IS AFTER THE SLOWDOWN PARAMETER. SPEEDUP IS HYBRID VS. SLOWDOWN-MATCHED GPU-ONLY BASELINE; VERDICT PER SECTION III-E.

Platform	Workload	Native r	Achieved r	Speedup	95% CI	CPU tasks (%)
MPS	cifar10_resnet20	–	–	–	–	–
	cifar10_row_gru	–	–	–	–	–
	california_mlp	–	–	–	–	–
	mnist_mlp	–	–	–	–	–
		–	–	–	–	–
T4	cifar10_resnet20	–	–	–	–	–
	cifar10_row_gru	–	–	–	–	–
	california_mlp	–	–	–	–	–
	mnist_mlp	–	–	–	–	–
		–	–	–	–	–

D. Experiment 3: Calibration Cache Amortization

TABLE VIII
CALIBRATION-CACHE AMORTIZATION ON THE SELECTED $N = 4$ RELATED CHECKPOINT SESSION. THE CASE IS SELECTED BY FILTERING EXPERIMENT 2 FOR HYBRID AFFINITY, PREFERING A NATIVE REGIME AND FALLING BACK TO A SLOWED ONE; THE OPERATING POINT ROW LABELS WHICH. $\text{RATIO} = \text{VANILLA_REFERENCE SESSION TOTAL} / \text{CACHED_COMPOSED SESSION TOTAL}$. BREAK-EVEN N^* IS ABSENT WHEN $T_v \leq T_p$.

Metric	Selected regime
Workload	—
Platform	—
Operating point (native/slowed)	—
rq3_config	—
Selected hetero-config tuple	—
Calibration cost (s)	—
vanilla_reference session, $N = 4$ (s)	—
cached_composed session, $N = 4$ (s)	—
Session speedup ratio (descriptive)	—
Break-even N^*	—
Amortization label	—

VI. DISCUSSION (TODO)

A. RQ1: Algorithm Redesign Applicability

- vectorization affinity: vmap ensembling pattern maps a leading dimension of same-architecture parameter states into batched tensor ops; flat/dense mlp rows closest match to intended vectorization shape
- architecture interpretation: row-gru has time dependence by design: each row update depends on the previous recurrent state, so vmap can batch perturbations but cannot remove inner sequence; resnet rows combine perturbation batching with convolution kernels whose GPU backend behavior already highly optimized
- composition: the compiled-vmapped comparison asks whether the composed path improves on the better individual transform under shared timing-verdict taxonomy

B. RQ2: Hybrid Scheduling Conditions

- explanatory variable: the CPU/GPU forward+loss inference-throughput ratio. Forward+loss inference dominates grid runtime (IV), so inference throughput ratio is the relevant narrowing of the general CPU/GPU throughput ratio (II-E).
- expected ratio threshold: near or above parity, hybrid scheduling absorbs useful CPU work; far below parity, GPU-only is expected to dominate or leave the verdict unresolved
- workload dependence: sequential row-gru should require less artificial GPU slowdown to approach inference parity than cifar10 resnet20; nvidia’s DL perf guide for CNNs [17] describes tensor-shape and alignment conditions under which conv kernels reach high GPU throughput; resnet20 forward path is dominated by 2d conv layers that meet those conditions; rowgrud is a recurrent sequence of

matrixvector ops with dependence between steps; hence expect resnet20h above rowgru on GPU

- platform interpretation: on a discrete-memory CPU/GPU system, CPU compute reads and writes host DRAM and contends with the GPU only at the PCIe bus, on host-device transfers. On a unified-memory system (Apple M4), CPU and GPU share one physical memory pool and one memory bus by design, so any concurrent CPU compute competes with the GPU for memory bandwidth regardless of transfers; expect hybrid scheduling to tolerate concurrent CPU workers more on the discrete-memory machine than on unified-memory

C. RQ3: Calibration and Cache Reuse

- measurement scope: single-measurement ratios are descriptive; CI based cross-session repeatability not measured
- cache point: calibration runs once at start; reuse only matters if per-checkpoint saved time amortizes one-time calibration cost
- negative case: if calibration resolves gpu-only then cache has no useful policy to reuse; report amortization refuted for that case

D. Hardware Dependence and Limits

- scoped to unified-memory consumer hardware and discrete vram free-tier cloud hardware
- four workloads span mostly orthogonal ML families but do not support claims beyond our workload set
- expected dependence axes: memory pressure for large vmap chunks, backend-specific handling of the transformed tensor program, scheduler overhead at small grid-task granularities, and platform-specific numerical behavior at surface validation
- observed differences should link to platform arch differences

VII. CONCLUSIONS (TODO)

summarize RQ answers and the practical recommendations following from measured applicability boundaries, including cases where evidence supports baseline instead of applying an optimization

VIII. REFERENCES

REFERENCES

- [1] I. J. Goodfellow et al., Qualitatively Characterizing Neural Network Optimization Problems. International Conference on Learning Representations (ICLR), 2015. arXiv:1412.6544. <https://arxiv.org/abs/1412.6544>
- [2] H. Li et al., Visualizing the Loss Landscape of Neural Nets. Advances in Neural Information Processing Systems 31 (NeurIPS), 2018. arXiv:1712.09913. <https://papers.nips.cc/paper/7875-visualizing-the-loss-landscape-of-neural-nets>
- [3] T. Xie et al., LossLens: Diagnostics for Machine Learning Through Loss Landscape Visual Analytics. IEEE Computer Graphics and Applications, 2025. arXiv:2412.13321. <https://doi.org/10.1109/MCG.2024.3509374>
- [4] T. Xie et al., LossLens code repository. GitHub, accessed May 22, 2026. <https://github.com/VADERASU/loss-lens-CGA>

- [5] A. Prakash et al., Visualizing Loss Landscapes with Adaptive Sampling and Gradient Free Slicing. Non-peer-reviewed course report, Brown University CS 237 (Interactive Data Visualization), 2024. <https://cs.brown.edu/courses/cs237/2025/proceedings2024.pdf>
- [6] C. P. Kruskal and A. Weiss, Allocating Independent Subtasks on Parallel Processors. *IEEE Transactions on Software Engineering*, SE-11(10), 1001–1016, 1985. <https://doi.org/10.1109/TSE.1985.231547>
- [7] C. Augonnet et al., StarPU: A Unified Platform for Task Scheduling on Heterogeneous Multicore Architectures. *Concurrency and Computation: Practice and Experience*, 23(2), 187–198, 2011. <https://doi.org/10.1002/cpe.1631>
- [8] Intel, Heterogeneous Execution (HETERO). OpenVINO documentation, accessed May 22, 2026. <https://docs.openvino.ai/2024/openvino-workflow/running-inference/inference-devices-and-modes/hetero-execution.html>
- [9] Intel, Automatic Batchng. OpenVINO 2023.3 documentation, accessed May 25, 2026. https://docs.openvino.ai/2023.3/openvino_docs_OV_UG_Automatic_Batching.html
- [10] Intel, OpenVINO auto-batch plugin source code. GitHub repository, version 2023.3.0, accessed May 25, 2026. https://github.com/openvinotoolkit/openvino/tree/2023.3.0/src/plugins/auto_batch/src
- [11] T. Goldstein et al., loss-landscape code repository. GitHub, accessed May 21, 2026. <https://github.com/tomgoldstein/loss-landscape>
- [12] PyTorch, torch.func.functional_call API docs, accessed May 21, 2026. https://docs.pytorch.org/docs/stable/generated/torch.func.functional_call.html
- [13] PyTorch, torch.vmap API docs, accessed May 21, 2026. <https://docs.pytorch.org/docs/stable/generated/torch.vmap.html>
- [14] PyTorch, torch.compile API docs, accessed May 25, 2026. <https://docs.pytorch.org/docs/stable/generated/torch.compile.html>
- [15] PyTorch, torch.allclose API docs, accessed May 25, 2026. <https://docs.pytorch.org/docs/stable/generated/torch.allclose.html>
- [16] PyTorch, Model ensembling tutorial, accessed May 21, 2026. <https://docs.pytorch.org/tutorials/intermediate/ensembling.html>
- [17] NVIDIA, Convolutional Layers User’s Guide, accessed May 26, 2026. <https://docs.nvidia.com/deeplearning/performance/dl-performance-convolutional/index.html>
- [18] K. Cho et al., Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014. arXiv:1406.1078. <https://doi.org/10.3115/v1/D14-1179>
- [19] J. Chung et al., Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. *arXiv.org preprint arXiv:1412.3555*, 2014. <https://arxiv.org/abs/1412.3555>
- [20] T. Kalibera et al., Rigorous Benchmarking in Reasonable Time. *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, 2013. <https://doi.org/10.1145/2464157.2464160>
- [21] T. Kalibera and R. Jones, Quantifying Performance Changes with Effect Size Confidence Intervals. *IEEE Transactions on Software Engineering*, 2020. <https://doi.org/10.1109/TSE.2020.3001339>
- [22] R. C. Whaley and J. J. Dongarra, Automatically Tuned Linear Algebra Software. *Proceedings of the 1998 ACM/IEEE Conference on Supercomputing (SC98)*, 1998. <https://doi.org/10.1109/SC.1998.10004>
- [23] J. Ansel et al., PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, 929–947, 2024. <https://doi.org/10.1145/3620665.3640366>
- [24] D. Crankshaw et al., Clipper: A Low-Latency Online Prediction Serving System. *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 613–627, 2017. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw>
- [25] T. Chen et al., TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 578–594, 2018. <https://www.usenix.org/conference/osdi18/presentation/chen>
- [26] H. Topcuoglu, S. Hariri, and M.-Y. Wu, Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3), 260–274, 2002. <https://doi.org/10.1109/71.993206>

IX. APPENDICES (TODO)